

Chapter 11: Introduction to MongoDB

Gregory S. DeLozier, PhD

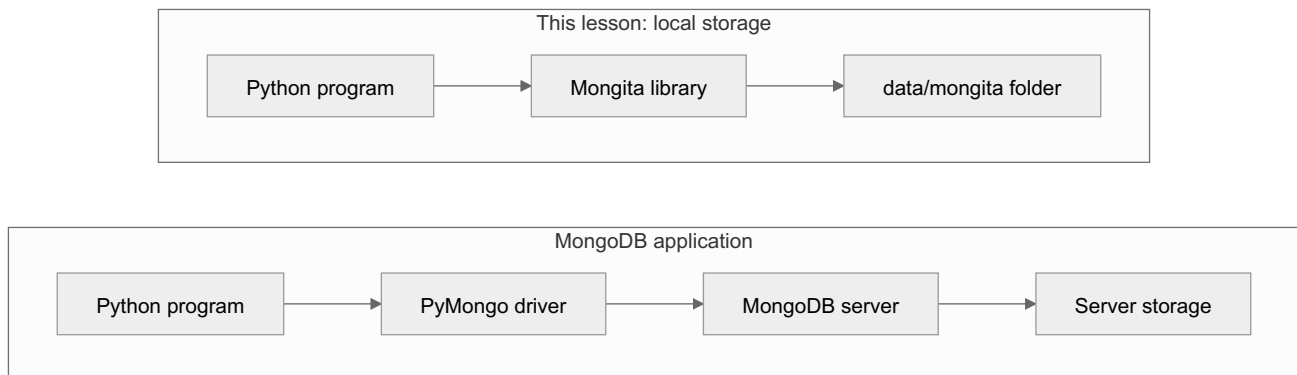
So far, we have stored application data in relational tables. Python gives us objects and dictionaries; our database layer translates between those values and rows. A document database gives us another way to organize the stored data. We can save a record with named fields, retrieve it as a dictionary, and work with those fields directly. Documents can also contain lists and other documents. (1)

We will begin with Mongita and the mongoose examples from our interactive session. Mongita runs inside Python and stores data locally. It implements a subset of the MongoDB/PyMongo application programming interface (API), so we can learn the basic operations without setting up a database server. PyMongo is the Python driver used to communicate with MongoDB. (2)

Where the Database Runs

With SQLite, the database engine runs inside our application. Mongita works that way too. In this chapter, `MongitaClientDisk` writes into a folder named `data/mongita` beside the example programs. We can exit Python and come back to the same records.

MongoDB uses a different arrangement: our Python program uses a client library to communicate with a database server. That server may be local or hosted elsewhere. Mongita lets us begin with familiar method calls, but it does not run a MongoDB server or implement every MongoDB feature. (2)



Both arrangements give us persistent data. The distinction here is where the database work runs. We will use one Python session at a time with the local lesson database.

Documents and Collections

Here are the two starting records:

```
{"name": "Meercat", "does_not_eat": "Snakes"}
{"name": "Yellow mongoose", "eats": "Termites"}
```

Each dictionary represents a **document**. The keys name its fields. Notice that both documents have name, but only one has eats. We did not create a table with a fixed set of columns before storing these records.

A **collection** holds documents. Our database is named `hello_world_db`, and its collection is named `mongoose_collection`.

Relational term	Document database term	In this example
Table	Collection	<code>mongoose_collection</code>
Row	Document	One mongoose record
Column	Field	<code>name</code> , <code>eats</code> , or <code>weight</code>
Primary key	<code>_id</code> field	Identifier for one document

These are useful comparisons, not identical storage rules. A document can have fields that its neighbors do not have. Our application still needs to decide what a record means and which fields it needs. For example, an animal without an `eats` field has no recorded value for that field; that does not tell us that it eats nothing.

MongoDB stores documents using BSON (Binary JSON), a binary representation of JSON-like documents. JSON means JavaScript Object Notation; BSON extends the available types to include values such as `ObjectId`. In Python, we will work with dictionaries and the corresponding Python value types. (3)

Set Up the Examples

Try This: From the course repository root, enter the example directory and install its dependencies in a virtual environment. The environment below lives in your home directory, outside the repository.

```
cd chapter-11-intro-to-mongo/code
python3 -m venv ~/venvs/advanced-database-mongita
source ~/venvs/advanced-database-mongita/bin/activate
python -m pip install -r requirements.txt
python setup_database.py
```

The setup program inserts the two starting documents when the collection is empty. Running it again leaves existing documents alone. To start the lesson over, use:

```
python setup_database.py --reset
```

Reset deletes all documents in this lesson's `mongoose_collection` and inserts the two starting records again. It does not reset other collections. Use it before beginning the sequence below if you have already experimented with this collection.

The repository ignores `data/` folders. Keep generated database files there so that your local work does not become part of a Git commit.

There is also a complete demonstration with printed results:

```
python mongita-session-clean.py --reset
```

That command resets the same collection and runs the whole sequence. For the hands-on version below, run `python setup_database.py --reset` afterward, then start Python:

```
python
```

The remaining **Try This** blocks run in this same Python session unless a block says otherwise. Copy the code, not output lines. The setup has already inserted the two starting records.

Open the Collection

Try This:

```
from lesson import connect
client, mongoose_collection = connect()
print(mongoose_collection.count_documents({}))
```

Output:

```
2
```

The `connect()` helper keeps the location consistent between the setup program, the interactive session, and the complete demonstration. Its database code is short:

```
from pathlib import Path
from mongita import MongitaClientDisk

DATA_DIR = Path(__file__).resolve().parent / "data" / "mongita"

DATA_DIR.mkdir(parents=True, exist_ok=True)
client = MongitaClientDisk(host=str(DATA_DIR))
hello_world_db = client.hello_world_db
mongoose_collection = hello_world_db.mongoose_collection
```

This excerpt is from a Python file, where `__file__` names that file. In the interactive session, use the helper rather than retyping the excerpt. Selecting a database and collection gives us handles through which to work; we do not need a `CREATE TABLE` statement.

The seed operation in `lesson.py` is:

```
mongoose_collection.insert_many([
    {"name": "Meercat", "does_not_eat": "Snakes"},
    {"name": "Yellow mongoose", "eats": "Termites"},
])
```

Do not run that insertion again during the sequence. Each insertion creates more documents, even if their names match existing names. The name field is ordinary data, not a unique key. The example records also appear in Mongita's introductory example. (2)

Retrieve a Document

Try This:

```
yellow = mongoose_collection.find_one({"eats": "Termites"})
print(yellow)
print(yellow["name"])
print(yellow["_id"])
```

`find_one()` returns a dictionary when it finds a match, or `None` when it does not. Use brackets to access a dictionary field: `yellow["name"]`. (4)

The dictionary includes the original fields and an automatically assigned `_id`. We did not supply that field when inserting the document. MongoDB normally uses an `ObjectId` for an automatically generated identifier, and Mongita does the same in this example. (3)

A filter such as `{"eats": "Termites"}` describes which documents we want. An empty filter, `{}`, places no conditions on the match.

Filter	Meaning in our example
<code>{}</code>	All documents
<code>{"name": "Meercat"}</code>	Documents with this name
<code>{"weight": {"\$gt": 1}}</code>	Documents whose numeric weight exceeds 1

These dictionaries play the role that a `WHERE` condition played in our Structured Query Language (SQL) queries. `$gt` means greater than. We will add `weight` next.

Update a Field

Try This:

```
result = mongoose_collection.update_one(
    {"name": "Meercat"},
    {"$set": {"weight": 2}},
)
print(result.matched_count, result.modified_count)
print(mongoose_collection.find_one({"name": "Meercat"}))
```

The counts are 1 1 on the first run after setup. The first dictionary selects the document. The second describes the change. \$set adds weight when it is absent or assigns a new value when it is present. The other fields remain. The result separates how many documents matched from how many changed. (5)

We added a field to one document. The Yellow mongoose document still has no weight. A numeric query for weights greater than 1 now finds Meercat and does not find Yellow mongoose.

A Cursor Is a Consumable Result

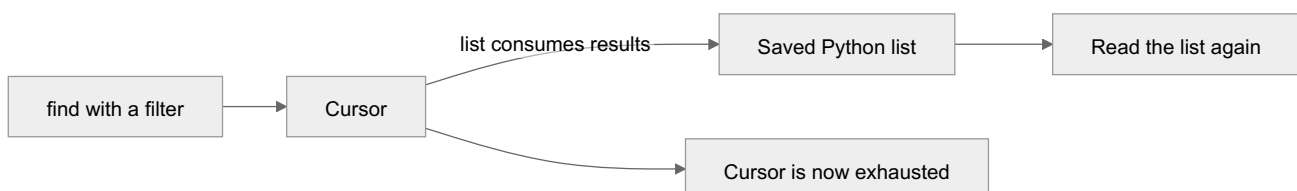
Try This:

```
cursor = mongoose_collection.find({"weight": {"$gt": 1}})
mongooses = list(cursor)
print(len(mongooses))
print(list(cursor))
print(len(mongooses))
```

Output:

```
1
[]
1
```

`find()` returns a **cursor**, which we can iterate to obtain matching documents. (4) The first `list(cursor)` consumes its results. Asking that same cursor for another list gets nothing because it has already reached the end. The saved mongooses list still contains the document. We can iterate that list again.



For a fresh query, call `find()` again. To keep a small result for repeated use, save it in a list. The list holds all those results in memory, so that choice matters when a query returns many documents.

Do not confuse this with `list(yellow)`. `yellow` is a dictionary, so iterating it produces its keys. It is already one document.

Try This:

```
print(list(yellow))
print(list(mongoose_collection.find({"weight": {"$gt": 300}})))
print(mongoose_collection.find_one({"weight": {"$gt": 300}}))
```

The first line prints field names. The second prints []; the third prints None. Both queries found no matching document, but their return types differ.

Carry an Identifier as Text

A web address or form field carries text. Our document's `_id` is an `ObjectId`. We can convert it to text for transport and construct an `ObjectId` from that text when looking up the document. The constructor accepts a valid 24-character hexadecimal representation. (6)

Try This:

```
from bson.objectid import ObjectId

identifier = str(yellow["_id"])
print(identifier)
found = mongoose_collection.find_one({"_id": ObjectId(identifier)})
print(found["name"])
```

The name is Yellow mongoose. The identifier itself will differ from one setup to another. Keep the conversion explicit so that our lookup uses the same value type as the stored field.

Delete and Insert Again

Deleting a document removes that record. Inserting another document with the same name creates a new record. The old fields do not come back unless we supply them.

Try This:

```
old = mongoose_collection.find_one({"name": "Meercat"})
result = mongoose_collection.delete_one({"_id": old["_id"]})
print(result.deleted_count)
print(mongoose_collection.find_one({"_id": old["_id"]}))
```

Output is 1, then None. `delete_one()` removes one matching document and reports the count. (7) Using `_id` identifies the particular document we read.

Try This:

```
inserted = mongoose_collection.insert_one(
    {"name": "Meercat", "does_not_eat": "Snakes"}
)
print(inserted.inserted_id != old["_id"])
print(list(mongoose_collection.find({"weight": {"$gt": 1}})))
print(mongoose_collection.count_documents({}))
```

Output:

```
True
[]
2
```

The replacement has a new identifier and no `weight` field. There are two documents again, but the database is not in the same state it was in before the deletion.

Verify Persistence

Try This: Close this Python session:

```
client.close()
exit()
```

From the same code directory, start python again. Do not run setup or reset between sessions. Then run:

```
from lesson import connect
client, mongoose_collection = connect()
print(mongoose_collection.count_documents({}))
print(mongoose_collection.find_one({"name": "Meercat"}))
client.close()
```

The count is still 2. The Meercat document is the one we reinserted, without `weight`. The Python variables from our earlier session are gone; the saved documents remain in the configured folder.

We now have the operations needed to build a database layer: create, read, update, and delete, usually abbreviated CRUD. Our application can continue to call functions such as `get_pet()` or `update_pet()`. Inside those functions, document operations can take the place of SQL statements.

Exercise: Extend the Collection

Start with `python setup_database.py --reset`, then open the collection in Python. Add three invented animal records. Give two of them a numeric weight; leave it out of the third. Keep the names distinct so that you can follow each record.

Write a filter that returns exactly two of your new records. Save the result in a list and print their names. Then change one weight with `update_one()` and run the same query again. Explain why the membership changed or stayed the same.

Delete one of your records by `_id`. Exit Python and reopen the database to verify which documents remain. Keep your commands in a Python file so that someone else can follow what you did. Decide whether that file is intended to continue existing work or start over; make any reset explicit.

References

1. Wikipedia contributors. “Document-oriented database.” https://en.wikipedia.org/wiki/Document-oriented_database
2. Scott Rogowski and contributors. “Mongita.” Project README, introductory example, and API summary. <https://github.com/scottrogowski/mongita>
3. MongoDB, Inc. “Documents.” MongoDB Manual. <https://www.mongodb.com/docs/manual/core/document/>
4. MongoDB, Inc. “Find Documents.” PyMongo Driver documentation. <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/query/find/>
5. MongoDB, Inc. “Update Documents.” PyMongo Driver documentation. <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/update/>
6. MongoDB, Inc. “objectid - Tools for working with MongoDB ObjectIds.” PyMongo documentation. <https://pymongo.readthedocs.io/en/stable/api/bson/objectid.html>
7. MongoDB, Inc. “Delete Documents.” PyMongo Driver documentation. <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/delete/>

Questions for Thought

1. In your current collection, which documents have no `weight` field? Inspect them. How would treating a missing weight as zero change the meaning of a report?
2. Save a query result as a list, update one of the matching documents, and run the query again. What does the saved list contain, and what does the new query return?
3. Insert two documents with the same name. How can you update exactly one of them? What should an application use as its identifier?
4. Where would you put a rule that every animal must have a name? How would that choice affect a second program that writes to the same database?
5. Which functions in our earlier database abstraction layer could keep their names and arguments when we replace relational storage with document storage? Which return values would need attention?